MOT Odometer Fraud Detection

Detecting mileage fraud in UK MOT records using unsupervised machine learning, no labelled data required.

Project Structure

```
🗀 project
│
├── fyp_final_file.ipynb      # Main notebook — run this
├── fyp_file_final.ipynb # Draft
├── new_fyp_file_(1).ipynb # Draft

├── Dataset # Dataset link
└── README.md
```

What is this project?

Every car in the UK needs an MOT test each year. During that test, the mileage on the odometer is recorded. Some dishonest sellers wind back (or "clock") the odometer to make a car look like it has done fewer miles than it actually has — this inflates the car's value and hides wear and tear.

This project uses machine learning to automatically spot suspicious vehicles in the official DVSA MOT dataset by looking at how a car's driving behavior changes over time. If a car has suddenly "driven much less" than it always has, that's a red flag.

Goal

Compare four unsupervised anomaly detection algorithms and find out which one is best at catching subtle odometer fraud:

Model	Type
Isolation Forest	Tree-based anomaly detection
One-Class SVM	Boundary-based detection
Local Outlier Factor (LOF)	Density-based detection
Elliptic Envelope	Statistics-based detection

Dataset

- **Source:** Official UK Government DVSA MOT test records (2018–2024)
- **Size:** 10% sample by vehicle → 40,000 vehicles randomly selected
- **Key columns used:** vehicle_id, test_date, test_mileage, make, fuel_type, first_use_date

The dataset is not included in this repo as it is publicly available from the DVSA open data portal. Place the file at MOT/mot_2018_2024_10percent_by_vehicle.csv.gz in your Google Drive.

How it works

Step 1 — Feature Engineering

Rather than looking at raw mileage numbers, we calculate behavioral features that describe *how* a car is being driven:

Feature	What it means
mileage_diff	Miles driven since the last MOT
miles_per_day	Average daily driving intensity
usage_trend_shift	How this year's driving compares to last year (the main fraud signal)
car_age_at_test	How old the car was at the time of testing

Step 2 — Fraud Injection (for testing only)

Since there are no real fraud labels in the dataset, we inject 600 fake fraudulent vehicles to test the models. Each fake fraud vehicle has its latest mileage set just slightly higher than the previous test — mimicking a "Clocked Commuter" who has had their odometer wound back but not to zero.

Step 3 — Data Cleaning

Before training, we remove obvious bad data using DVSA-inspired rules:

- Remove vehicles driving more than 300 miles/day (physically impossible for normal use)
- Remove entries where driving intensity dropped by more than 50% with no explanation
- Remove any rows with negative mileage

Step 4 — Train the Models

All four models are trained only on clean, normal vehicles. They learn what "normal" looks like, then flag anything that doesn't fit.

Step 5 — Test & Evaluate

The models are tested on a separate forensic dataset containing:

- The 600 injected fraud vehicles
- 2,000 real normal vehicles (to test false alarm rate)
- And all negative mileage difference vehicles are also included in the testing dataset to check if the model can catch actual frauds.

Results

Model	Accuracy	Precision	Recall	F1-Score
One-Class SVM	95.07%	91.79%	90.09%	90.93%
Isolation Forest	89.48%	86.77%	72.38%	79.17%
Local Outlier Factor	77.11%	72.83%	26.55%	38.92%

Elliptic Envelope	80.49%	82.30%	36.86%	50.91%

One-Class SVM it caught 9 out of every 10 fraudulent vehicles while keeping false accusations low.

🚀 How to Run

This project runs on Google Colab (recommended) as it uses Google Drive for data storage.

1. Upload the dataset to Google Drive

Place the MOT CSV file here:

My Drive/MOT/mot_2018_2024_10percent_by_vehicle.csv.gz

2. Open the notebook in Google Colab

Upload fyp_final_file.ipynb to Colab or open it directly from your Drive.

3. Install dependencies

All required libraries are pre-installed in Colab. If running locally, install them with:

bash
pip install pandas numpy scikit-learn matplotlib seaborn matplotlib-venn joblib

4. Run all cells in order

The notebook is structured from top to bottom just click Runtime → Run all.

Libraries Used

- pandas — data loading and manipulation
- numpy — numerical operations
- scikit-learn — machine learning models and evaluation
- matplotlib / seaborn — visualisations
- matplotlib-venn — Venn diagram for model comparison

- joblib — saving and loading trained models

Requirements

- Python 3.8+
- Google Colab (recommended) or Jupyter Notebook
- ~2GB RAM minimum for the 40,000 vehicle sample

Key Concepts

What is a Clocked Commuter? A fraudulent vehicle that has had its odometer partially roll back. The mileage still goes up slightly, but much less than the vehicle's own historical average making it hard to spot by eye.

What is Recall? Out of all the actual frauds, how many did the model catch? A recall of 90% means it caught 9 out of 10.

What is Precision? Out of all the vehicles the model flagged as fraud. How many were actually fraudulent? High precision means fewer innocent drivers are falsely accused.

Author

Irzam Latif

University of Hertfordshire

Final Year Project — Mse DataScience
Dataset: DVSA MOT data — UK Open Government Licence